Deep Learning and Its Applications (CO3133) – Assignment 2

Deep Learning on Large-Scale Data and Specialized Tasks

Semantic Segmentation of Urban Street Scenes: A Baseline and a Pretrained Model Compared

Le Pham Tien Long[1,2(✉)] Ngo Tieu Nghi[1,2] Ho Minh Nhat[1,2]

[1] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet, Dien Hong Ward, Ho Chi Minh City, Vietnam.
[2] Vietnam National University Ho Chi Minh City (VNU-HCM), Linh Xuan Ward, Ho Chi Minh City, Vietnam.
Instructor: Le Thanh Sach **Semester:** 261 **Group:** 18 – DeepDive
{long.lephamtien, nghi.ngotieu, nhat.hominh}@hcmut.edu.vn

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Problem Formulation

- **Task:** semantic segmentation of urban street scenes, one prediction *per pixel*.
- **Input:** an RGB frame $x \in \mathbb{R}^{3\times H\times W}$.
- **Output:** per-pixel logits $z = f_\theta(x) \in \mathbb{R}^{19\times H\times W}$; the prediction is $\hat{y}_{ij} = \arg\max_c z_{cij}$, so one 512×1024 frame carries $524{,}288$ predictions.
- **Research question:** with the data, the split and the protocol fixed, how much of the gap between a from-scratch model and a pretrained one comes from *pretrained features*, and how much from architecture, loss or capacity?
- **Dataset:** Cityscapes fine annotations [1].
- **Main difficulty:** the classes span almost three orders of magnitude in pixel share, and the output must keep spatial resolution all the way through.

Cityscapes

- **Source:** recorded from a vehicle in 27 cities; per-pixel polygons drawn by expert annotators [1].
- **Size:** 5,000 fine frames at 1024×2048; 11 GB images, 808 MB labels.
- **Licence:** non-commercial research and education, account required.
- **Classes:** 34 label ids, **19 evaluated**; the rest map to an ignore label excluded from loss and metrics.
- **Unlabelled:** 11.8% of pixels, of which 7.7% is the bonnet and rectification border.
- **Bias:** daylight, fair weather, European cities, one camera.



Figure 1: Training frames and their label maps.

Pixel Distribution Decides the Metric

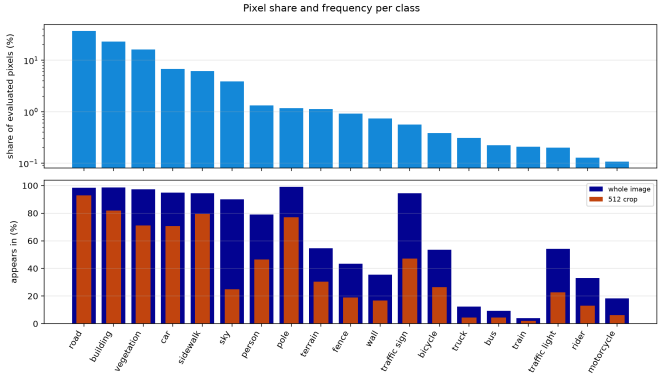


Figure 2: Pixel share and frequency per class.

- **Three classes dominate:** road, building and vegetation cover **75.8%** of evaluated pixels.
- Predicting *only* those three already reaches about **0.76 pixel accuracy**.
- **So mIoU is primary** and pixel accuracy decides nothing: mIoU averages over classes, so the other sixteen cannot be hidden.
- Per-class IoU is always reported beside the mean.

Two Kinds of Scarcity, One Remedy Will Not Fix Both

Table 1: Two kinds of scarcity.

	Class	Pixels	Images	Area when present	Crops
Thin	pole	1.17%	99%	0.83%	77%
	traffic sign	0.56%	94%	0.31%	47%
Rare but large	train	0.21%	**4%**	0.91%	2%
	bus	0.22%	**9%**	0.66%	4%

- **Thin classes** are never short of examples: poles appear in 99% of images. They are lost to *boundary imprecision*.
- **Rare-but-large classes** cover 3 to 4× their average share *when present*, but are seldom seen at all.
- **Cropping sharpens the second:** trains fall from 4% of images to 2% of crops. Sky, present in 90% of images, falls to 25% of crops because it sits in a band.
- **A reweighted loss addresses the second, not the first**, which is why the ablation varies the loss.

Splitting: the Unit Is the City

Table 2: Split sizes.

Ours	Source	Cities	Images
Train	official train	15	2,406
Val	official train	3	569
Test	official val	3	500

- **Why the city:** consecutive frames of one street share buildings, parked cars and road surface. Splitting by *frame* would put near-duplicates on both sides.
- **Why not the official test:** its labels are withheld, so the official *validation* partition serves as the test set.
- The three cities that *select* the checkpoint therefore come out of train.
- **Leakage controls:** split committed as a file, not regenerated; normalization never from scored data; test read once, after selection.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Pipeline

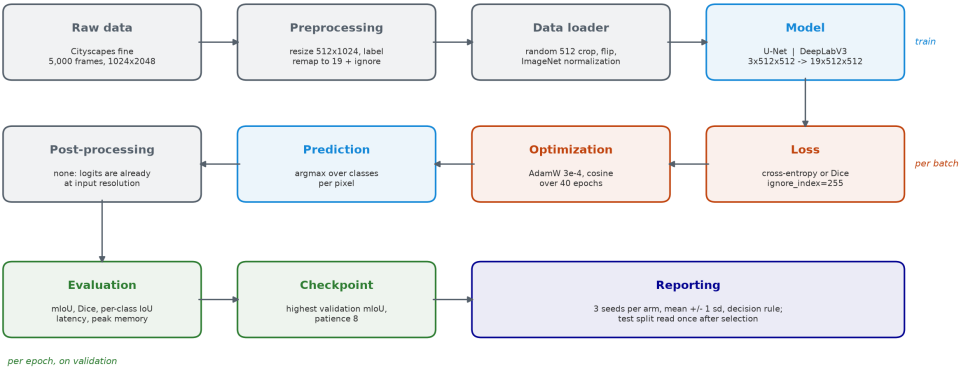


Figure 3: The pipeline.

- **Post-processing is empty by design:** both models emit logits at input resolution, so the prediction is an `argmax`; nothing is upsampled or smoothed.
- **Evaluation runs on two terms:** validation every epoch selects the checkpoint; test is read *once*, after selection.

U-Net, the Baseline

Written from scratch [2]; no pretrained weights. Base width 32, depth 4, $7{,}763{,}635$ parameters.

$$e_\ell = \mathrm{DoubleConv}(\mathrm{MaxPool}_2(e_{\ell-1}))$$
$$d_\ell = \mathrm{DoubleConv}([\,\mathrm{ConvT}_2(d_{\ell+1});\, e_\ell\,])$$

- Widths double, $32 \to 64 \to 128 \to 256$, bridged at 512.
- Convolutions carry **no bias**: each is followed by batch normalisation, which makes it redundant.

The skip connection is the architecture

Four poolings take a 512-pixel crop to **32 pixels**, enough context to say *what* a region is, far too coarse to say *where* its boundary runs.

Each decoder stage concatenates the encoder map of its own resolution, putting the upsampled semantics back onto the detail that produced them.

DeepLabV3, the Pretrained Model

Torchvision DeepLabV3 [3] + MobileNetV3-Large [4], pretrained on a COCO subset over an ImageNet backbone. $11{,}024{,}963$ parameters.

- **No decoder:** dilated kernels keep the feature map at stride 8, and atrous spatial pyramid pooling gathers context at several rates in parallel.
- **Head replaced:** the published 1×1 convolution predicts 21 classes; ours predicts 19. Backbone and pooling module transfer unchanged.

Two details that matter

Auxiliary head removed, not computed and discarded, so training pays nothing for an output no loss reads.
Normalization taken from the weights, not assumed. Feeding a differently normalised batch to a pretrained backbone degrades it *silently*, with no error to notice.

Fine-tuning procedure: one stage. The final convolution is replaced, then *every* parameter trains from epoch 1 at one learning rate, with no gradual unfreezing and no per-layer rates. This is the same procedure the U-Net receives, so the comparison is between *models*, not training schedules.

Losses and Metrics

Cross-entropy, over labelled pixels only:

$$\mathcal{L} = -\frac{1}{|\Omega|}\sum_{(i,j)\in\Omega} \log \frac{\exp(z_{y_{ij}ij})}{\sum_c \exp(z_{cij})}$$

$\Omega = \{(i,j) : y_{ij} \neq 255\}$. Without it, the network would learn to reproduce the bonnet of the recording vehicle.

Soft Dice [5], the ablation arm:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{C}\sum_{c=1}^{C} \frac{2\sum_\Omega p_{cij}\mathbf{1}[y_{ij}{=}c] + \varepsilon}{\sum_\Omega p_{cij} + \sum_\Omega \mathbf{1}[y_{ij}{=}c] + \varepsilon}$$

- **What they weight:** cross-entropy averages over *pixels*, so a class counts in proportion to the ground it covers. Dice averages over *classes*, so a pole counts as much as the road.
- **Metrics:** mIoU primary; Dice, per-class IoU, pixel accuracy secondary. All from one running 19×19 confusion matrix, because a validation pass holds 298M pixels, so masks are never concatenated.
- mIoU is a `nanmean`: a class absent from the ground truth is *excluded*, not scored zero.

What Was Written, and What Was Imported

Table 3: Attribution.

Component	Origin
U-Net, Dice loss, confusion matrix and every metric	written by the group
Training loop, evaluation, experiment runner, figures	written by the group
DeepLabV3 and its pretrained weights	`torchvision.models.segmentation`
Layer primitives, cross-entropy, AdamW, cosine schedule	PyTorch [6]
labelId $\rightarrow$ trainId table	`cityscapesscripts`
Plots	matplotlib

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

One Protocol for Every Arm

Table 4: Shared training protocol.

Setting	Value
Input	resize 512×1024, random 512^2 crop
Optimizer	AdamW [7], 3×10^{-4}, wd 0.05
Batch size	8 (301 steps/epoch)
Epochs	40 max, cosine over 40
Early stop	patience 8 on validation mIoU
Checkpoint	highest validation mIoU
Seeds	0, 1, 2 per configuration
Hardware	Apple M4 Pro, 14 cores, 48 GB, MPS
Software	Python 3.12.0, PyTorch 2.14.0
Precision	full `float32`, no AMP

Search effort is zero, and equal

No hyperparameter search was run. Every arm uses the same settings, fixed before training, and changes **exactly one** of them.
Every score is therefore a score *under a common protocol at an untuned configuration*, not the best that configuration could reach.

Reproducibility limit, reported not hidden

The Apple GPU backend has no deterministic kernel for the scatter-add DeepLabV3 reaches when it interpolates logits back to full resolution. U-Net reproduces bit-exactly; DeepLabV3 diverges by $\sim 4 \times 10^{-7}$ per step.

Four Experiments, Each Varying One Key

Table 5: The experiment matrix.

Role	Factor varied	Hypothesis	Se
Main comparison	U-Net from scratch vs. pretrained DeepLabV3	pretrained wins at equal capacity	3
Controlled experiment	backbone frozen vs. fully fine-tuned	full fine-tune wins; widest on thin classes	
Ablation	cross-entropy vs. Dice loss	Dice lifts train, bus, truck	
Extension	MobileNetV3 vs. ResNet50 backbone	ResNet50 wins at $3.6\times$ params	

Decision rule, fixed before any result was seen

A beats B *only* when its mean validation mIoU is higher **and** the two intervals of mean $\pm$ one standard deviation across three seeds **do not overlap**.

Overlapping intervals $\Rightarrow$ **indistinguishable**, which is itself a result and is *not* resolved by picking the higher mean. A claim about one class applies the same rule to that class's IoU. One seed supports no claim of superiority.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. **Results**
5. Error Analysis
6. Conclusion

Main Comparison

Table 6: Main comparison.

Configuration	Params	s/ep	h	ms@1	GB	Val mIoU	Test mIoU
DeepLabV3-ResNet50	39,638,355	739.9	8.22	72.3	4.37	*0.6665*	*0.7076*
DeepLabV3-MobileNetV3	11,024,963	140.4	1.52	15.2	3.41	$\mathbf{0.6136 \pm 0.0051}$	$\mathbf{0.6180 \pm 0.0049}$
frozen backbone	8,053,011	95.3	0.84	14.7	3.41	0.4961 ± 0.0049	0.5003 ± 0.0088
Dice loss	11,024,963	150.5	0.50	14.9	3.41	0.4958 ± 0.0045	0.4983 ± 0.0123
U-Net, from scratch	7,763,635	260.2	2.89	22.6	6.57	0.4836 ± 0.0154	0.4702 ± 0.0071

- **One large effect, three ways of losing it.** Only the fully fine-tuned pretrained model separates. The other three three-seed arms are **mutually indistinguishable**, within 0.0126 of each other; the frozen and Dice arms differ by **0.0003**.
- **Pixel accuracy ranks them the other way round:** U-Net 0.9202 beats frozen 0.8959 and Dice 0.8982, while losing on mIoU. Exactly what the EDA predicted.

Decision Rule Applied

Table 7: Every pairwise comparison, on validation mIoU.

Comparison	Verdict	Margin
DeepLabV3 vs. U-Net	DeepLabV3 better	0.1301
DeepLabV3 vs. frozen backbone	Full fine-tuning better	0.1175
DeepLabV3 vs. Dice loss	Cross-entropy better	0.1178
U-Net vs. frozen backbone	**Indistinguishable**	0.0126
U-Net vs. Dice loss	**Indistinguishable**	0.0123
Frozen vs. Dice loss	**Indistinguishable**	0.0003
Anything vs. ResNet50	Indicative only, one seed	n/a

Why three seeds were necessary

The U-Net's seed spread is 0.0154, **three times** that of the pretrained arms, with seeds at 0.4687, 0.4824, 0.4995. Its *best* seed would have tied the frozen arm; its worst would have lost clearly. A one-seed study could have reported either outcome.

Training Behaviour

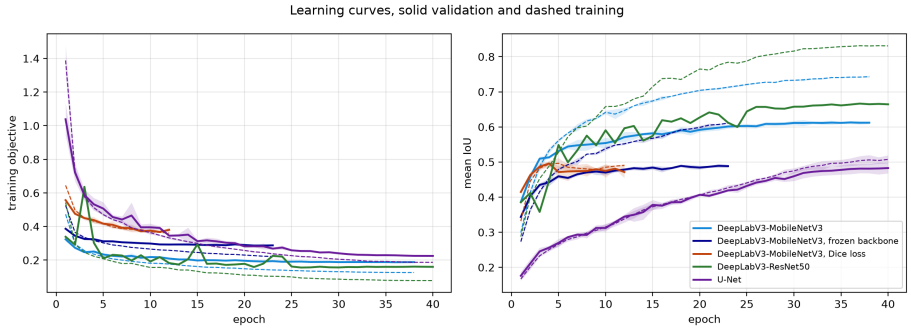


Figure 4: Solid validation, dashed training; the left axis mixes two objectives.

- **Budget was right for DeepLabV3:** best epoch 30/31/35 of 38/39/40, flat from about epoch 32.
- **The U-Net is undertrained:** best epoch 38/40/39 of 40, still rising at the last epoch. **0.4836 is a floor**, so the margin over it is an upper bound.
- **Dice peaked at epoch 4** on all three seeds and stopped at 12.

Where the Difference Appears

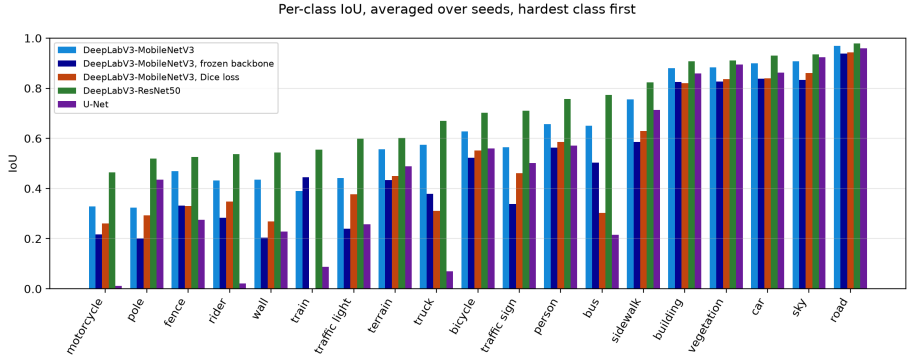


Figure 5: Per-class IoU on test, hardest class first.

- **Frozen loses on all 19 classes**, but very unevenly: 0.027 on road, 0.044 on car against 0.215 on truck, 0.213 on traffic sign and 0.173 on traffic light. A frozen backbone keeps what fills a street scene and loses what does not.
- **Pretraining is not uniformly better.** The from-scratch U-Net *beats* DeepLabV3, with separated intervals, on **pole** (0.4126 vs. 0.3277), sky and vegetation.

Compute Cost

The U-Net is Pareto-dominated

30% *fewer* parameters than DeepLabV3, yet

- $1.9\times$ the seconds per epoch,
- $1.5\times$ the latency at batch 1, $2.7\times$ at batch 8,
- $1.9\times$ the peak memory (6.57 vs. 3.41 GB),
- and 0.130 lower mIoU.

Parameter count is **not** a proxy for cost: its convolutions run at full resolution and its decoder holds every encoder map alive.

Freezing is a training-only saving

0.84 h against 1.52 h, a 45% saving, for 0.118 mIoU.
Smaller than it sounds: **73% of parameters stay trainable**, because the ASPP head holds 8.05M against the backbone's 2.97M.
Inference cost is identical (14.7 vs. 15.2 ms), because the same forward pass runs either way.

Scaling the backbone

ResNet50: $+0.0529$ val mIoU for $5.3\times$ s/epoch and $4.8\times$ latency. One ResNet50 run costs more than the nine MobileNetV3 runs combined.

Qualitative Results

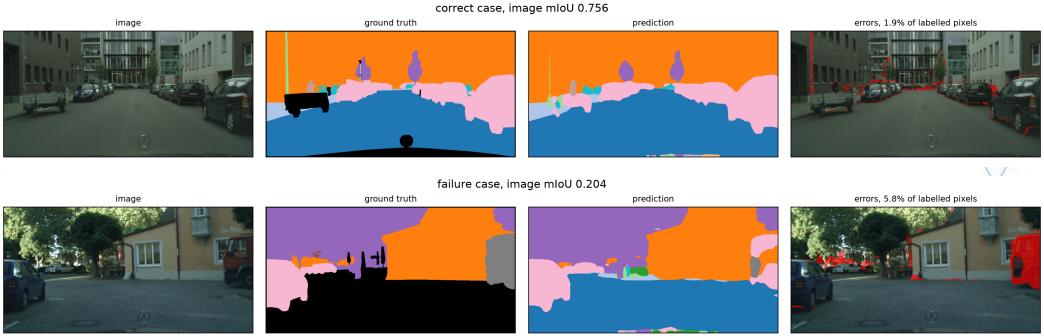


Figure 6: Best image (0.756 mIoU) above, worst (0.204) below.

- **Correct case:** the error is a thin outline tracing every boundary, so the model knows *what*, and is imprecise about *where*.
- **The failure case has only 5.8% of labelled pixels wrong**, *below* the 6.7% split average, yet is the worst image in the split. Most of the courtyard carries the ignore label, so few pixels are scored and each class has tiny support. **Per-image mIoU and per-image error are only loosely coupled.**

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Error Taxonomy

Table 8: Error taxonomy, as a share of each model's own errors.

Group	DeepLabV3	ResNet50	Frozen	Dice	U-Net
Thin structures	24.7%	23.0%	19.6%	21.9%	18.7%
Upright surfaces	24.0%	26.5%	24.2%	24.0%	23.4%
Participant vs. background	17.5%	15.7%	16.5%	17.7%	15.8%
Ground surfaces	14.6%	14.8%	20.3%	17.4%	17.1%
Traffic participants	6.5%	6.8%	6.3%	6.4%	**12.5%**
Other	12.7%	13.3%	13.1%	12.7%	12.4%
Error rate	6.7%	**4.9%**	10.4%	10.2%	8.0%

The error structure is a property of the task, not of the model

Error rates differ by more than $2\times$ across the five configurations, yet the composition is nearly identical. Improving the model lowers the total and leaves the proportions alone.

The Groups, and What Causes Them

- **Thin structures, 24.7%.** Pole $\rightarrow$ building is the single largest flow in the matrix (0.91M pixels, 5.9% of all errors); pole $\rightarrow$ vegetation adds 3.6%. Pole is the hardest of the 19 classes (0.3230 IoU) *despite appearing in 99% of images*, so this is a **resolution failure, not data scarcity**. A stride-8 feature map upsampled bilinearly cannot represent a structure a few pixels wide.
- **Upright surfaces, 24.0%.** Building $\leftrightarrow$ vegetation, 1.16M and 1.13M pixels, *nearly symmetric*, which is the signature of a boundary error rather than a bias. Wall and fence add a taxonomy problem: low barriers separated from building by convention, not appearance.
- **Ground surfaces, 14.6%.** The road/sidewalk kerb is a few pixels high and often occluded by parked cars. Grows to 20.3% for the frozen arm.
- **Traffic participants, 6.5%, and the one place a model differs in *kind*.** 12.5% for the U-Net, nearly double. It scores **0.0205 on rider** against DeepLabV3's 0.4316. Rider and person differ only in what the person sits on, which requires recognising the vehicle beneath. **This is where pretrained features help most.**

What would improve it

Five of six groups are dominated by *boundary* pixels $\Rightarrow$ the lever is **resolution**, not the objective. Train and evaluate at native 1024×2048; add a full-resolution decoder path. **Not** a reweighted loss: the ablation measures that directly, and it makes things worse.

Ablation: Cross-Entropy vs. Dice Loss

Hypothesis (pre-registered): Dice raises IoU on train, bus and truck, which are rare but large when present, because it weights classes rather than pixels.

Table 9: The ablation, on validation IoU.

Class	Cross-entropy	Dice	Change	P(absent)
train	0.6007 ± 0.0383	$\mathbf{0.0000 \pm 0.0000}$	-0.6007	85.1%
bus	0.4154 ± 0.0923	0.1944 ± 0.1696	-0.2211	70.0%
truck	0.3797 ± 0.0398	0.1952 ± 0.0233	-0.1845	69.3%
motorcycle	0.4661 ± 0.0064	0.2775 ± 0.0313	-0.1886	60.2%
traffic sign	0.5518 ± 0.0036	0.4692 ± 0.0068	-0.0826	0.6%
pole	0.3277 ± 0.0085	0.3048 ± 0.0072	-0.0229	0.0%
road	0.9722 ± 0.0004	0.9502 ± 0.0016	-0.0220	0.0%

Refuted, and in the opposite direction

Dice is worse on *all nineteen* classes. Correlation between P(absent) and the damage Dice does: $\mathbf{r = -0.809}$.

Why Dice Fails Here

The mechanism

For a class with *no* positive pixels in the batch:

- **Cross-entropy** gives it no term of its own; it is only a competitor in the softmax denominator.
- **Dice** gives it a full term $\dfrac{0+\varepsilon}{\sum_{\Omega} p_{cij} + 0 + \varepsilon}$, maximised by driving that class's probability to **zero everywhere**.

Dice **actively suppresses** every class missing from the current eight crops.

Measured

10.43 of 19 classes appear in a given 512^2 crop; 15.92 in a batch of 8.
So **16.2%** of Dice terms in a typical step are suppression terms.
For *train*, present in 2% of crops: roughly **85% of the 301 steps per epoch push its logit down**, about 45 push it up.
$\Rightarrow$ the only exact zero in this report: under Dice the model never predicts a single train pixel, on any seed, on either split.

It also explains the curve: peaked at epoch 4 on all three seeds, train-to-validation gap -0.002. Not overfitting, not underfitting: the suppression terms balance the overlap terms and the objective stops producing useful gradient.
What this does *not* establish: that class balancing is useless here. Accumulating Dice over an epoch, or masking absent classes out of the mean, avoids this exact mechanism. Neither was run.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. **Conclusion**

Answering the Research Question

How much of the gap between a from-scratch model and a pretrained one comes from pretrained features rather than architecture, loss or capacity?

A sharp answer

Fully fine-tuned pretrained features are worth

- $\mathbf{0.1301}$ validation mIoU over the from-scratch U-Net,
- $\mathbf{0.1175}$ over the *same weights* with the backbone frozen,
- $\mathbf{0.1178}$ over the *same model* trained with a class-balanced objective,

and those three alternatives are **mutually indistinguishable**, within 0.0126 of one another. One factor accounts for essentially the whole spread.

An unexpected qualification

The advantage is **not uniform**. The from-scratch U-Net is better, with separated intervals, on **pole, sky and vegetation**, because a full-resolution decoder localises thin structures that a stride-8 backbone cannot. Five of six error groups are boundary-dominated.
Pretraining supplies recognition rather than resolution.

Carried forward: **DeepLabV3-MobileNetV3, fully fine-tuned**, at $\mathbf{0.6180 \pm 0.0049}$ test mIoU and 0.7441 ± 0.0048 test Dice, 140 s/epoch, 15.2 ms/image.

Limitations

- **The U-Net is undertrained.** Validation mIoU still rising at epoch 40 on all three seeds, so 0.4836 is a floor and the margin over it an *upper bound*.
- **The model comparison confounds two factors.** Its arms differ in architecture *and* in pretraining; no run here separates them. A DeepLabV3 trained from scratch would.
- **The backbone extension is one seed** and supports no claim of superiority.
- **Three seeds estimate a standard deviation from three observations.** "Indistinguishable" means *not separated at this sample size*, not equal.
- **The data is narrow**: daylight, fair weather, European cities, one camera, and images were resized to a quarter of their pixels for compute, itself a ceiling on the thin classes that dominate the errors.
- **Reproducibility has a hardware limit.** The Apple GPU backend has no deterministic kernel for the interpolation DeepLabV3 uses, so rerunning a DeepLabV3 seed will not reproduce its number exactly. The reported spread includes that nondeterminism.

What Comes Next

In the order the evidence supports them:

1. **Resolution.** Train and evaluate at the native 1024×2048, or add a full-resolution decoder path (DeepLabV3+). This attacks the thin-structure and boundary groups, which are most of the error, and the 512×1024 resize was a choice made for compute rather than accuracy.
2. **The clean version of the model comparison.** A DeepLabV3 trained from scratch separates pretraining from architecture in a single run.
3. **The class-balancing question the ablation *reopened*.** Dice accumulated over an epoch, or masking absent classes out of its mean, avoids the suppression mechanism measured here.

Two practices that changed the conclusions

Reporting per-class numbers beside the mean, and fixing the decision rule before seeing results. The per-class table located the one setting in which the *weaker* model wins, and the rule turned three apparently different outcomes into a single tie.

Thank You

References I

[1] M. Cordts et al., "The cityscapes dataset for semantic urban scene understanding," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[3] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[4] A. Howard et al., "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[5] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *Fourth International Conference on 3D Vision (3DV)*, 2016.

References II

[6] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[7] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.